

Booking Platform API — Running Guide

1. Prerequisites

- Node.js 20+ and npm
- Docker & Docker Compose (recommended — no local PostgreSQL install needed)
- Alternatively: a local PostgreSQL 16 instance

2. Quick Start (Docker)

```
git clone <repo-url> cd booking-  
platform-api cp .env.example .env  
docker-compose up --build
```

This starts PostgreSQL, waits for it to become healthy, runs **prisma migrate deploy** automatically, then starts the API on port 3000.

3. Manual Setup (Local PostgreSQL)

```
npm install cp .env.example  
.env  
  
# edit .env → set DATABASE_URL to point to your local PostgreSQL npx prisma migrate dev --name  
init  
  
npm run prisma:seed # optional: loads sample user + services npm run start:dev
```

4. Environment Variables (.env)

Variable	Description	Example
PORT	Port the API listens on	3000
DATABASE_URL	PostgreSQL connection string	postgresql://user:pass@localhost:5432/db
JWT_ACCESS_SECRE	T Signs short-lived access tokens	a-long-random-string
JWT_ACCESS_EXPIR	ES_INAccess token lifetime	15m
JWT_REFRESH_SECR	ETSigns refresh tokens (must differ)	a-different-long-string

5. Running Database Migrations

```
npx prisma migrate dev --name <migration-name> # development npx prisma migrate  
deploy # production / CI npx prisma studio # visual DB browser (optional)
```

6. Verifying the API is Running

- Base URL: <http://localhost:3000/api/v1>
- Swagger UI: <http://localhost:3000/docs>
- Health check: `GET /api/v1/health` → returns database connectivity status

7. Seeded Test Credentials

```
Email: admin@en2h.com Password:  
Password123!
```

Running `npm run prisma:seed` (or the Docker flow) creates a sample account:

Two sample services are also seeded, so bookings can be created immediately without manual setup.

8. Testing the Endpoints

- Import `postman/Booking-Platform-API.postman_collection.json` into Postman
- Use the interactive Swagger UI at `/docs` (supports bearer-token auth directly in the browser)

9. Running Tests

```
npm run test # unit tests  
npm run test:cov # unit tests with coverage report
```

10. Troubleshooting

- **Cannot connect to database** — confirm PostgreSQL is running and `DATABASE_URL` in `.env` is correct.
- **Prisma Client not generated** — run `npx prisma generate`.
- **Port already in use** — change `PORT` in `.env` or stop the process using port 3000.
- **401 on protected routes** — ensure the Authorization header is `Bearer <accessToken>` from `/auth/login`.